arXiv:2508.04462v1 [cs.LG] 6 Aug 2025

CARD: Cache-Assisted Parallel Speculative Decoding for Efficient Large Language Model Inference

Enyu Zhou[1], Kai Sheng[1], Hao Chen[2], Xin He[1]*
[1]Guangzhou Institute of Technology, Xidian University
[2]CSEE, Hunan University
zhouenyu@stu.xidian.edu.cn, kaisheng@xidian.edu.cn, haochen@hnu.edu.cn, hexin@xidian.edu.cn

Abstract

Speculative decoding (SD), where an extra draft model first provides multiple draft tokens and the original target model then verifies these tokens in parallel, has shown great power for LLM inference acceleration. However, existing SD methods must adhere to the 'draft-then-verify' paradigm, which forces drafting and verification processes to execute sequentially during SD, resulting in inefficient inference performance and limiting the size of the draft model. Furthermore, once a single token in the candidate sequence is rejected during the drafting process, all subsequent candidate tokens must be discarded, leading to inefficient drafting. To address these challenges, we propose a cache-based parallel speculative decoding framework employing a 'query-and-correct' paradigm. Specifically, CARD decouples drafting and verification: the draft model generates candidate tokens to populate a shared cache, while the target model concurrently rectifies the draft model's generation direction. This effectively enables the target model to perform inference at speed approaching that of the draft model. Our approach achieves up to 4.83× speedup over vanilla decoding without requiring fine-tuning of either the draft or target models. Our code is available at https://github.com/hunzhizi/CARD.

1 Introduction

Modern large language models (LLMs) like ChatGPT-o1 and DeepSeek-R1(Guo et al., 2025) demonstrate exceptional capabilities across various domains. However, these models with hundreds of billions of parameters face significant latency challenges during autoregressive generation when deployed. Autoregressive decoding frameworks rely on sequential token generation, where each new token is conditioned on all prior outputs and every

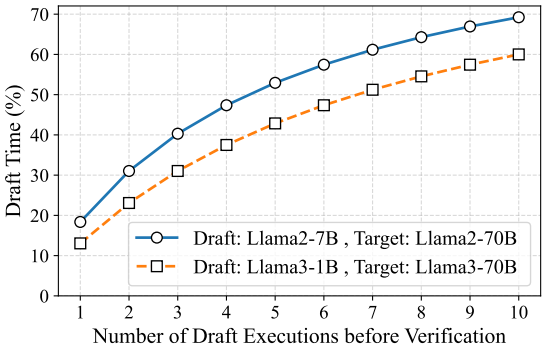

Figure 1: The ratio of draft model execution time to total inference time.

decoding step must access the full model parameters. This creates a serious bottleneck for practical applications. To address this limitation, Speculative Decoding (SD) frameworks with their 'draft-then-verify' paradigm have emerged as promising solutions(Leviathan et al., 2023; Chen et al., 2023; Sun et al., 2023). The core idea involves using a lightweight draft model to generate r candidates, which are subsequently verified by the larger target model in a single forward pass. While SD achieves lossless acceleration, its efficiency is fundamentally constrained by two key factors: **(1) fixed-length chain-based drafting structures limit average acceptance length and may lead to wasted computation when rejected**; **(2) sequential dependency between drafting and verification phases results in hardware underutilization, especially for large-scale target models.** As shown in the Figure 1, a noticeable mutual waiting problem exists. As r increases, the draft model consumes an increasingly larger proportion of total inference time. Consequently,when r becomes too large, the target model spends more than half its time waiting for drafting results. This severely wastes GPU resources as the draft model occupies computational resources that could be used by the target

*Corresponding author

model.PEARL(Liu et al., 2025) proposes a parallel strategy using 'pre-verify' and 'post-verify' mechanisms to address the selection problem of parameter r. However, its adoption of a chain structure results in inefficient drafting, causing the target model to miss numerous opportunities for acceleration.

Existing approaches attempt to improve target model mean acceptance length through structural innovations. SpecInfer(Miao et al., 2024) introduced tree-based attention mechanisms to enable parallel verification of multiple candidate sequences, improving acceptance rates. The introduction of tree attention is essentially due to the memory-bound nature of target model execution. Moderately increasing the number of input tokens does not affect target model execution efficiency. Approaches like EAGLE(Li et al., 2024),Medusa(Cai et al., 2024),Ouroboros(Zhao et al., 2024) and Opt-Tree(Wang et al., 2025) achieve extreme performance acceleration by constructing large token trees. However, this is impractical in actual deployment environments where target model computational resources are precious, making it nearly impossible to construct large token trees for verification. This introduces a new problem: **while token tree verification achieves higher acceptance rates, it places a tremendous burden on the target model.** Excellent approaches like EAGLE and Medusa that train draft models for better inference also inevitably sacrifice drafting capabilities due to the mutual waiting problem. For instance, EAGLE employs a draft model with a parameter size equivalent to only one decoder layer of the target model., while Medusa uses a single MLP layer for prediction. With a better parallel strategy, we could use draft models more flexibly and achieve better performance. Furthermore, draft models and target models have different computational characteristics, and placing them on the same device does not fully utilize hardware resources. Draft models have smaller parameter counts and require fewer computational resources. While the target model is awaiting the draft model's output, the allocated computational resources remain idle, leading to significant hardware underutilization.

This paper introduces CARD, a novel speculative decoding framework that fundamentally rethinks the interaction paradigm between draft and target models by parallelizing them. Our main contributions include the following.

Parallel Execution Framework: We break the sequential dependency in traditional SD through a cache-supported architecture that allows draft and target models to perform inference simultaneously, implementing a new "query-and-correct" paradigm (query the cache; select paths with the highest confidence; correct draft model's generation direction).

Adaptive Caching Mechanism: A dynamic cache structure helps adaptively determine effective drafting length while maximizing token acceptance rates by preserving historical states.

Computation Offloading from Target to Draft Model: We maintain chain-based verification efficiency in the target model while achieving tree-like performance through the correction phase.

2 Preliminary

2.1 Generation Space

The generation space $\mathcal{G}$ is formally defined as the Cartesian product of token distributions across all decoding steps $t = 1$ to T:

$$\mathcal{G} = \prod_{t=1}^{T} \Delta(\mathcal{V}) \subset \mathbb{R}^{|\mathcal{V}| \times T},$$

where $\Delta(\mathcal{V})$ denotes the probability simplex over the vocabulary $\mathcal{V}$. This high-dimensional space encapsulates all possible output trajectories, with each point $\mathbf{g} \in \mathcal{G}$ representing a unique sequence of token probability distributions. For language models, $\mathcal{G}$ is implicitly parameterized by the model architecture and dynamically shaped through decoding algorithms (e.g., greedy sampling, nucleus sampling).

2.2 Generation Direction

For sequences already generated by the LLM, we describe them as sequences obtained by sampling along the Generation Direction.

3 Method

In CARD, we introduce a novel caching mechanism that breaks the traditional 'draft-then-verify' paradigm, enabling the target model to perform continuous inference. As shown in the Figure 2, CARD abandons the conventional 'draft-then-verify' approach in favor of a new 'query and correct' paradigm. CARD runs the draft and target models in parallel. The 'query and correct' approach accelerates the target model by caching its generation space and rectifying the generation direction of the draft model after verification. This

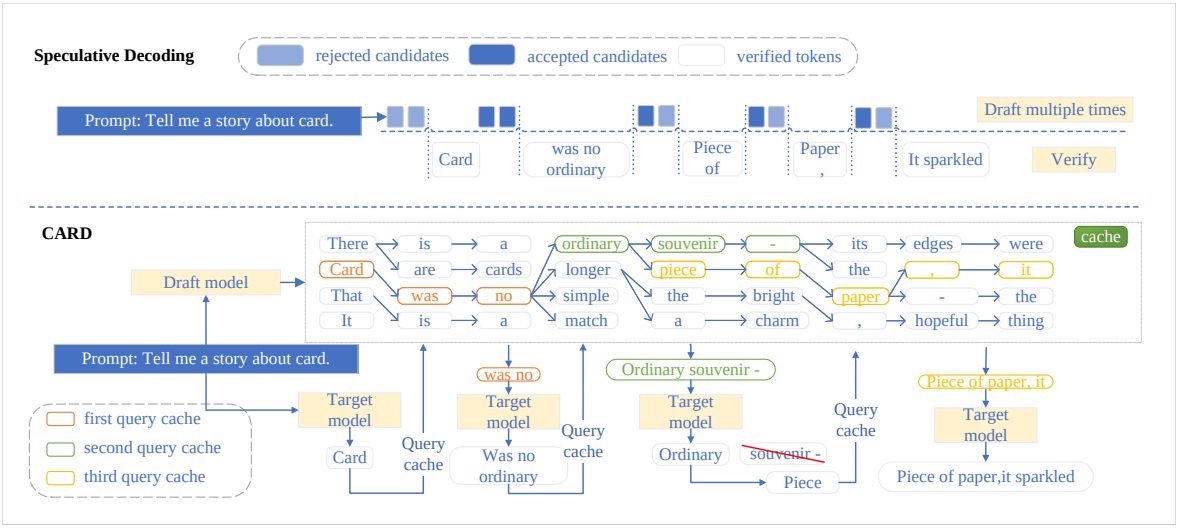

Figure 2: Overview of CARD.

represents a generalizable new paradigm that could potentially leverage database retrieval or other methods to cache operations for the target model.

However, in CARD, we follow the consistent strategy within the speculative decoding domain by employing a draft model to implement caching for the target model. CARD generates candidates for the target model through continuous drafting by the draft model, selecting a subset of these candidates as cache for the target model. Before each execution, the target model queries the cache and proceeds with inference carrying only the highest-weighted candidate token sequence. After inference, based on the verified tokens, CARD corrects the generation direction of the draft model, enabling it to continuously produce high-quality tokens.

3.1 Building the Cache

In CARD, we implement target model cache construction through continuous inference from the draft model. By leveraging the property that the draft model's output distribution is similar to the target model's (which has been proven in EAGLE-2 and OPT-tree), our draft model uses a tree attention mechanism to generate candidate tokens and selects a subset of these candidates to populate the cache. This approach is necessary because LLM vocabularies are extremely large. Take the Llama3 series as an example, where the vocabulary size is 128,256. Storing all possible outputs is impractical for two key reasons: 1) It would impose enormous memory overhead, and 2) Even for the draft model, inputting 128,256 tokens simultaneously is impossible. Additionally, when the number of input tokens becomes excessive, it may lead to compute-bound situations that slow down the draft model's inference. Experimental evidence demonstrates that caching a sufficiently small number of tokens is adequate to achieve good mean acceptance length.

We define the primary cache as $C^{(l)} = \{s_1^{(l)}, \ldots, s_K^{(l)}\} \subseteq \mathcal{V}^l$. We also construct a **secondary cache, $\mathcal{B}^{(l)}$, to store the candidate tokens generated from the primary cache along with their corresponding path scores.** The primary cache size is K, where $K = |C^{(l)}|$, l represents the current cache layer, and $\mathcal{V}$ denotes the vocabulary.

The cache construction process is illustrated in Figure 3. The draft model simultaneously processes all sequences from the current primary cache $C^{(l)}$ and computes the probability distributions over the next token in a single forward pass:

$$\{P_i^{(l)}\}_{i=1}^{K} = D_\phi(\text{TreeDecoding}(C^{(l)})) \quad (1)$$

For each distribution $P_i^{(l)}$, we select the top-k tokens to form the extension set:

$$T_i^{(l)} = \text{TopK}(P_i^{(l)}) = \{(y_{i,j}^{(l)}, p_{i,j}^{(l)})\}_{j=1}^{k} \quad (2)$$

where $p_{i,j}^{(l)}$ represents the conditional probability of token $y_{i,j}^{(l)}$. To populate the secondary cache, the score of each new candidate is calculated by weighting its conditional probability with the cumulative probability of its parent sequence:

$$\forall (i,j), w_{i,j}^{(l)} = p_{i,j}^{(l)} \cdot \sigma(s_i^{(l)}) \quad (3)$$

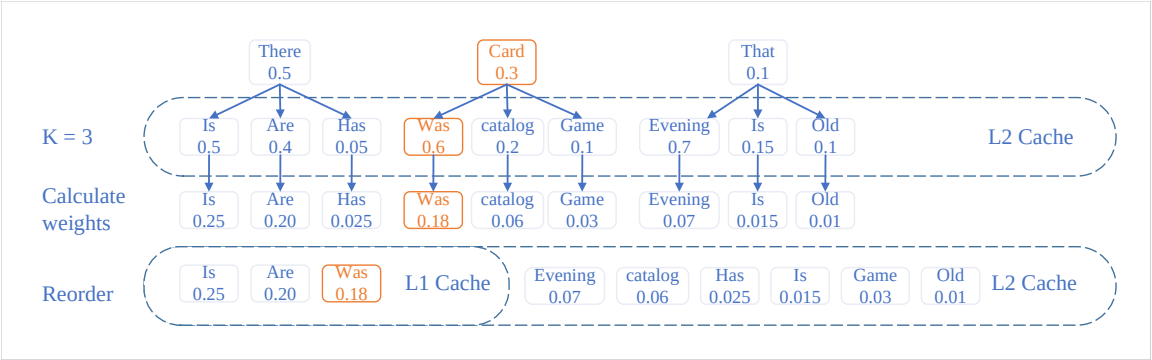

Figure 3: The process of constructing the cache.

where $\sigma(s_i^{(l)})$ denotes the cumulative probability of the parent sequence $s_i^{(l)}$:

$$\sigma(s_i^{(l)}) = \prod_{t=1}^{l} p_\theta(s_i^{(l)}[t] \mid s_i^{(l)}[1:t-1]) \quad (4)$$

The secondary cache must contain the candidate token, its full path score, and the index of its parent sequence to trace its origin. It is formally defined as a set of tuples:

$$\mathcal{B}^{(l)} = \{(y_{i,j}^{(l)}, w_{i,j}^{(l)}, i)\}_{i=1,j=1}^{K,k} \quad (5)$$

After constructing the secondary cache, we reorder all $K \times k$ candidates globally based on their path scores. From this sorted pool, we select the top-K candidates to form the new sequences for the next layer and update the primary cache. This ensures that only the most promising hypotheses are propagated. The process is defined as follows:

$$S^{(l)} = \text{Sort}(\mathcal{B}^{(l)}, \text{key=score}) \quad (6)$$

From the sorted list $S^{(l)}$, we select the top-K candidates. Let the m-th best candidate be the tuple (y_m^*, w_m^*, i_m^*), where y_m^* is the token, w_m^* is its full path score, and i_m^* is the index of its parent sequence in $C^{(l)}$:

$$\{(y_m^*, w_m^*, i_m^*)\}_{m=1}^{K} \subseteq S^{(l)} \quad (7)$$

We then form the new sequences by appending each winning token to its corresponding parent sequence:

$$s_m^{(l+1)} = s_{i_m^*}^{(l)} \oplus y_m^* \quad \text{for } m = 1, \ldots, K \quad (8)$$

Finally, the new primary cache for layer $l+1$ is formed by this set of K new sequences:

$$C^{(l+1)} = \{s_1^{(l+1)}, \ldots, s_K^{(l+1)}\} \quad (9)$$

3.2 Mask Construction for Draft Model

In CARD, the draft model generates a tree of candidate token sequences. Efficient inference for these candidates requires a precisely constructed attention mask for each drafting step. As illustrated in the Figure 4,this mask ensures that each token being generated attends only to its valid causal history within the dynamically expanding token tree, thereby maintaining the integrity of the auto-regressive generation process for the draft model.

The construction of this attention mask at each k draft step, for a set of N parallel candidate tokens, adheres to the following principles.

Each candidate token must be able to attend to the KV cache entries corresponding to its unique ancestral path within the token tree. This is achieved by inheriting and appropriately indexing the attention rights from its parent node in the tree. Specifically, the portion of the mask corresponding to previously generated candidates (from steps 1 to k-1) is derived from the mask of the parent candidate, effectively allowing the current token to "see" what its parent could see in the established tree structure.

In addition to the broader ancestral path, each candidate token is explicitly allowed to attend to the KV cache entry of its direct parent token from the immediately preceding drafting step (k-1). This creates a direct contextual link to the token from which it was branched.

For the N tokens being concurrently generated at the current drafting step k, a causal mask (e.g., a lower triangular matrix if tokens are ordered, or an identity matrix for simultaneous independent proposals before scoring) is applied. This ensures that a token's prediction is based only on preceding tokens within the current batch and itself, upholding the auto-regressive property within the current

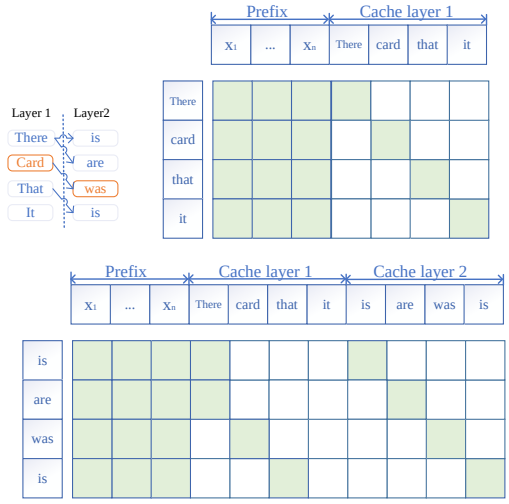

Figure 4: An example of mask construction.

generation step.

The final attention mask for step k is a concatenation of these components, dynamically growing in sequence length to accommodate the expanding KV cache from previous candidates in the tree and the current set of candidates. This systematic construction allows the draft model to efficiently evaluate multiple future possibilities in parallel while respecting the necessary information flow constraints.

3.3 Cache Correction

The purpose of the correction phase is to align the draft model's generation direction with the target model's generation direction, preventing divergence in generation direction. After the target model completes its verification, the validated path is passed to the draft model. The draft model then updates the information in its cache based on the tokens verified by the target model. This process involves pruning tokens from incorrect paths in the cache to free up computational resources, allowing for higher quality cache generation in the correct generation space direction. This approach enables the target model to perform inference at speed approaching that of the draft model when using an appropriate draft model. Moreover, this approach enables similar performance even without implementing tree attention mechanisms for parallel verification of multiple sequences in the target model, we can achieve similar performance. This is because one of the correction capabilities is to exclude incorrect candidate sequences. As a result, CARD is able to achieve similar or even stronger speedup with significantly less computation on the target model compared to other methods. The key innovation lies in the efficient alignment of generation spaces between the draft and target models through this correction mechanism.

4 Experiments

This section presents a systematic evaluation of CARD across diverse text generation benchmarks, with particular emphasis on its computational efficiency and performance stability across different architectural configurations.

4.1 Experimental Settings

Datasets. For code generation, we validate CARD on HumanEval(Chen et al., 2021) and MBPP(Austin et al., 2021) benchmarks. HumanEval contains 164 entries, each consisting of a text prompt and a prefix of a Python function. The MBPP test set contains 500 entries, where the entire function needs to be predicted given a text prompt and test cases. We randomly sampled 100 entries from MBPP for our test set.

For arithmetic reasoning, multi-round conversation, and multilingual tasks, we evaluate our method on GSM8K, MT-Bench, and MGSM(Shi et al., 2022). We randomly sampled 100 entries from GSM8K(Cobbe et al., 2021). For all tasks, the maximum number of new tokens to generate was set to 512.

Model Configurations. For HumanEval and MBPP, we tested the following model combinations: First group: Llama-2-chat-7B as the draft model and Llama-2-chat-70B as the target model(Touvron et al., 2023). Second group: Qwen-2.5-7B as the draft model and Qwen-2.5-70B as the target model(Yang et al., 2024). Third group: Llama3.2-1B as the draft model and Llama3.2-70B as the target model(Grattafiori et al., 2024).

For GSM8K, MT-Bench(Zheng et al., 2023), and MGSM, we used the first and third groups of models for testing. This approach allows us to demonstrate the impact of different draft models on the CARD architecture.

Evaluation Methods. We selected baselines comparable to our training-free method. We compared against vanilla autoregressive decoding, speculative decoding, lookahead decoding(Fu et al., 2024), Ouroboros, and PEARL, using the default configurations for these frameworks. In CARD,

Table 1: Performance comparison across different tasks and architectures

Task	Framework	Llama3 70B/1B		Llama2 70B/7B		Llama2 70B/7B Temp=1.0	
		tokens/s	speedup	tokens/s	speedup	tokens/s	speedup
GSM8k	Vanilla	8.75	1	8.80	1	8.80	1
	Speculative	21.22	2.42	16.68	1.89	16.57	1.88
	Lookahead	-	-	13.52	1.53	13.65	1.55
	Ouroboros	-	-	23.45	2.66	19.27	2.19
	PEARL[1]	25.76	2.94	20.34	2.27	18.74	2.12
	CARD	**41.34**	**4.72**	**29.81**	**3.39**	**20.30**	**2.30**
MGSM	Vanilla	9.48	1	9.64	1	9.64	1
	Speculative	20.32	2.14	17.21	1.78	17.14	1.77
	Lookahead	-	-	14.31	1.48	14.45	1.49
	Ouroboros	-	-	25.02	2.59	23.97	2.48
	PEARL	28.68	3.02	22.88	2.37	21.00	2.17
	CARD	**37.93**	**4.00**	**30.56**	**3.17**	**26.50**	**2.74**
MT-Bench	Vanilla	9.18	1	9.38	1	9.38	1
	Speculative	17.73	1.93	14.89	1.59	16.78	1.79
	Lookahead	-	-	14.02	1.49	14.01	1.49
	Ouroboros	-	-	23.68	2.52	21.84	2.32
	PEARL	22.97	2.50	20.27	2.16	18.99	2.02
	CARD	**28.32**	**3.08**	**25.97**	**2.76**	**25.20**	**2.68**

to adhere to our 'query and correct' paradigm, the communication ratio is defined as the ceiling of the ratio between the forward computation times of the target and draft models. This approach aims to maximize the draft model's inference speed and minimize wasted inference time. In all experiments, for 7B models, we set the communication ratio between our draft model and target model to 5:1, meaning the draft model executes 5 times for every single communication with the target model (where the target model queries the cache and corrects the draft model's generation direction) and $K = 50$. For 1B draft models, we set the communication ratio between draft model and target model to 7:1 and $K = 100$. Unless otherwise specified, the temperature was set to 0 for all methods.

Metrics.Speedup Ratio: The acceleration compared to vanilla autoregressive decoding.

Mean acceptance length: The average number of tokens accepted by the target model in each inference step. In CARD, the mean acceptance length is limited by the drafter's ability to predict the target model's generation direction and its own execution speed.

Hardware and Implementation. Experiments are conducted on 3× NVIDIA A800 80GB GPUs with NVLink ×8 interconnect and Intel Platinum 8350C CPU. To ensure cross-framework compatibility, we standardize experiments using HuggingFace Transformers v4.31.0 with automatic model parallelism. The temperature is set to 0 by default, unless otherwise specified, with greedy decoding employed for all experiments.

4.2 Overall results

Multi-model Performance: As shown in the Figure 5, CARD consistently outperforms other frameworks across models of varying sizes. CARD achieves a maximum speedup of 4.83× on Llama3 1B/70B models. However, when using the 7B model as the draft model, we did not observe more impressive acceleration ratios, which are limited by the inference speed of the draft model itself. (In the Appendix, we provide inference speeds for models of different sizes.) Nevertheless, CARD maintains superior performance compared to other frameworks thanks to its cache component, which enables it to achieve higher mean acceptance length.

As shown in Table 1, CARD achieves the highest speedup ratios across all tested architectures and tasks, demonstrating significant advantages over existing training-free acceleration frameworks.

State-of-the-art acceleration performance: CARD outperforms the vanilla autoregressive baseline by 4.72× (GSM8k), 4.00× (MGSM), and 3.08× (MT-Bench) with the Llama3 architecture, and consistently maintains the best speedup ratios under Llama2 configurations (3.39×/3.17× for GSM8k/MGSM). These improvements are particularly notable compared to the second-best framework PEARL(2.27×/2.37×), indicating CARD's superior algorithmic efficiency.

Temperature-robust Speedup: Across the

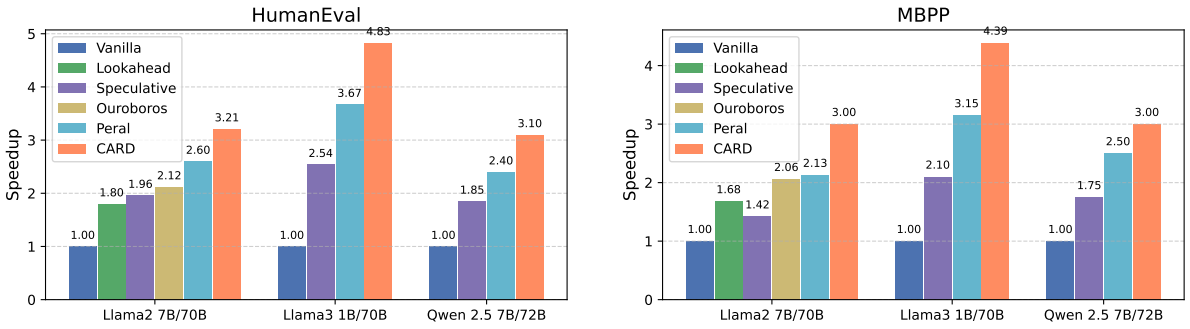

Figure 5: The greedy decoding speedup on HumanEval and MBPP

Table 2: The ablation studies of each component in CARD on GSM8k using Llama3 1B/70B

Method	tokens/s
Baseline	8.75
+ Cache (query but don't correct)	30.23
+ Correct (correct generation direction)	41.34

three datasets shown in the Table 1, CARD exhibits varying degrees of performance decline when temperature = 1. However, CARD still maintains a lead over other frameworks, demonstrating its unique capability to balance generation quality and acceleration under diverse sampling conditions.

Superior Performance with Smaller Draft Models: CARD achieves better acceleration when using Llama3-1B as the draft model, reaching a maximum speedup of 4.72×. This confirms the effectiveness and superiority of CARD's "query and correct" architecture. While the 7B-sized draft model is limited by its execution speed, the 1B model offers faster execution. Simultaneously, the presence of the cache ensures a high acceptance length for the target model. This combination results in better overall performance when using the 1B-sized model. It is worth noting that CARD is a training-free method, and we believe there is potential for even better alternatives in the cacher choices in future work.

4.3 Ablation studies

As shown in Table 2,we conducted ablation experiments on the GSM8k dataset using Llama3-1B/70B to evaluate our framework components. When only the cache component is added, the target model can perform queries but does not correct the draft model. This configuration reveals inherent instability in the draft model's directional prediction capability, where prolonged generation sequences exacerbate discrepancies between predicted and verified token distributions. Nevertheless, the inherent parallelism and abundant candidate tokens provided by the cache still enable achieving a reasonable speedup ratio. After incorporating the correction component, the phenomenon of generation direction prediction divergence was significantly mitigated, leading to further performance improvements in the framework.

4.4 The effectiveness of cache size (K)

As shown in the Figure 6 and Figure 7, we investigated the effect of parameter K on performance across MT-Bench and GSM8k datasets. We fixed the communication ratio at 7 and incrementally increased K from 5 by steps of 25, examining changes in mean acceptance rate, tokens per second, and cache hit rate. The results demonstrate that on both MT-Bench and GSM8k, Mean Acceptance Length increases with larger K values, indicating that larger cache sizes provide greater benefits to the target model. However, when K exceeds 50, the improvement becomes gradually less pronounced. Interestingly, speedup exhibits a pattern of initial growth followed by decline, suggesting that after the cache size reaches a certain threshold, the draft model becomes compute-bound. This subsequently affects the overall speed. We hypothesize that adjusting the communication ratio could be a strategy to mitigate this decline. The cache hit rate increases with K value, reaching an impressive maximum of 97%. This high hit rate validates the effectiveness of our correction process, which ensures the draft model consistently predicts along the target model's generation direction, enabling the target model to achieve such exceptional cache hit rates.

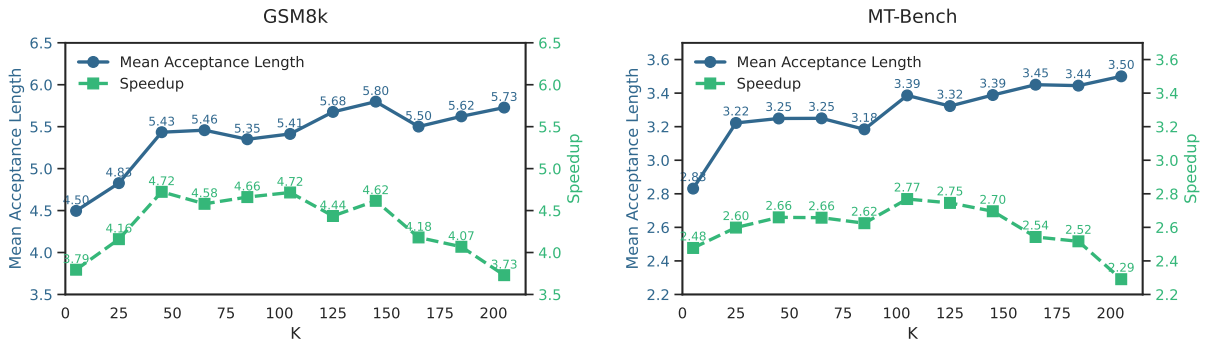

Figure 6: Variation in mean acceptance length and tokens per second with K

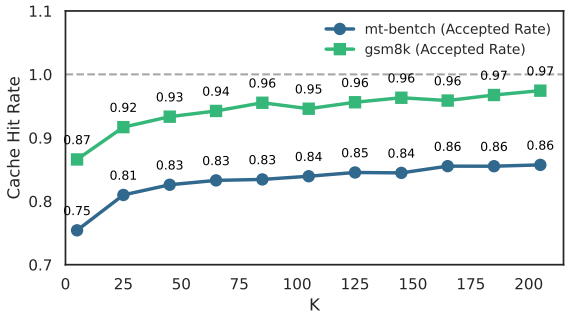

Figure 7: Cache hit rate variation with K

Table 3: Comparison of computational efficiency on GSM8k with 70B target models.EAGLE-3 and OPT-Tree use fixed tree sizes: 50 for OPT-Tree and 48 for EAGLE-3. In the draft model comparison, OPT-Tree uses a 7B model, while EAGLE-3 and CARD use 1B models.

Method	Target Model Computation	Draft Model Computation	Speedup
OPT-Tree	50×70 (3500)	7×50 (350)	1.9
EAGLE-3	48×70 (3360)	**1×10 (10)**	4.43
CARD	**7×70 (490)**	1×100 (100)	**4.72**

4.5 Comparing with Tree-Verified Methods

CARD achieves performance comparable to tree-verified methods through the 'query-and-correct' paradigm, while significantly reducing computational resources by offloading computation to the draft model, thereby freeing computational resources for the target model. We compared several different tree-based models in the GSM8k dataset by using theoretical complexity $Params \times L_{new}$, where $Params$ is the size of the model and L_{new} is the length of the input tokens. As shown in the Table 3,CARD achieves similar effects to EAGLE-3(Li et al., 2025) while utilizing substantially less computation on the target model. Although the length of candidate sequences generated per decoding step in CARD is constrained by the cache capacity, the communication ratio is 7, meaning the maximum average computational load is $(communicationRatio \times modelParameterSize)$. However, there are inevitably some losses during the correction process, making the actual value less than 7, 7 is used for illustrative purposes here.

5 Related Work

Large language model (LLM) inference acceleration has been extensively studied through diverse approaches including non-autoregressive decoding frameworks (Guo et al., 2020; Ghazvininejad et al., 2019), hardware-aware implementations (Dao et al., 2023; Kwon et al., 2023), and model compression techniques (Zafrir et al., 2019; Jiao et al., 2019). Drafting-then-verifying methods (Stern et al., 2018; Xia et al., 2022) achieve lossless acceleration through speculative sampling, with variants exploring small draft models (Leviathan et al., 2023; Zhou et al., 2023), internal target-model mechanisms (Cai et al., 2024; Fu et al., 2024), and external phrase retrieval (Saxena, 2023; He et al., 2023), while tree-style verification (Miao et al., 2024; Cai et al., 2024) enables parallel draft validation. Hardware optimizations focus on memory-efficient attention computation (Dao et al., 2023) and distributed execution (Shoeybi et al., 2019), and compression approaches reduce computational demands through quantization (Frantar et al., 2022), pruning (Han et al., 2015), and distillation (Hinton et al., 2015). These orthogonal directions—decoding algorithms, implementation optimizations, and model compression—are fre-

quently combined to maximize acceleration gains.

6 Conclusion

In this paper, we propose a novel paradigm called 'query-and-correct' for speculative decoding. We address the mutual waiting problem inherent in traditional 'draft-then-verify' approaches through our parallel strategy. By introducing a cache as middleware between the draft model and the target model, we enable the draft model to effectively predict the target model's generation space, successfully offloading computational resources from the target model to the draft model while maintaining high mean acceptance length. With these improvements, CARD achieves up to 4.83× acceleration completely training-free.

Limitations

While our cache-based architecture successfully addresses the mutual waiting problem, it does require additional GPU resources to cache the generation space for the target model. Although this doesn't introduce substantial computational overhead—for example, in an 8-GPU cluster setup, we allocate one GPU for caching operations—this approach is less suitable for resource-constrained environments. The 'query-and-correct' architecture is more appropriate for scenarios where computational resources are abundant and maximum acceleration is the priority. Furthermore, as our method is training-free, the selection of draft models may not be optimal. The query-and-correct architecture likely has greater potential through improved draft model selection and integration with LLM serving systems.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and 1 others. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-decoding for long-context inference. https://example.com/flash-decoding.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*.

Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-predict: Parallel decoding of conditional masked language models. *arXiv preprint arXiv:1904.09324*.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Junliang Guo, Linli Xu, and Enhong Chen. 2020. Jointly masked sequence-to-sequence model for non-autoregressive neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 376–385.

Song Han, Jeff Pool, John Tran, and William Dally. 2015. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D Lee, and Di He. 2023. Rest: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. 2019. Tinybert: Distilling bert for natural language understanding. *arXiv preprint arXiv:1909.10351*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*.

Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. 2025. PEARL: Parallel speculative decoding with adaptive draft length. In *The Thirteenth International Conference on Learning Representations*.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, and 1 others. 2024. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949.

Apoorv Saxena. 2023. Prompt lookup decoding.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, and 1 others. 2022. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31.

Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. 2023. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36:30222–30242.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and 1 others. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. Opt-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. 2022. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. *arXiv preprint arXiv:2203.16487*.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. 2019. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pages 36–39. IEEE.

Weilin Zhao, Yuxiang Huang, Xu Han, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. 2024. Ouroboros: Speculative decoding with large model enhanced drafting. *arXiv e-prints*, pages arXiv–2402.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, and 1 others. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623.

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2023. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*.

A Appendix

A.1 Execution Speed of Models with Different Sizes

Table 4: Throughput comparison of Llama models under transformers v4.31.0

Model	Tokens/sec
Llama-2-70B	9
Llama-2-7B	40
Llama-3-70B	9
Llama-3-1B	63

As shown in 4,our tests were conducted on transformers v4.31.0 (which originally doesn't support Llama-3). We modified the source code to enable Llama-3 inference. The implementation details and full benchmarking code are available in our open-source repository .

A.2 Why use cache?

Tree decoding techniques are widely employed in SD(speculative decoding) frameworks such as Medusa, Eagle series, SpecInfer, and various other SD frameworks. However, tree decoding imposes significant computational overhead on the target model. We know that computational resources for target models are scarce in actual deployment scenarios and can hardly sustain the substantial additional computational costs incurred by tree decoding. Meanwhile, draft models, with their smaller parameter size and lower computational overhead, often have more available computational resources.**We sought a method that could achieve effects similar to tree decoding but without its massive computational costs**. CARD addresses this challenge by constructing a cache and implementing a 'query and correct' strategy that offloads computation from the target model to the draft model. This approach maintains a high mean acceptance length while dramatically reducing computational requirements.

The existing serial execution strategy of 'draft-then-verify' significantly limits the performance ceiling of SD, as the target model must wait for the draft model's inference to complete before proceeding with its next inference cycle. CARD introduces cache as middleware, enabling parallel operation of the draft model and the target model, thereby opening up new possibilities for SD. Furthermore, in the verification phase, once a candidate is rejected, all subsequent candidates must be discarded, wasting historical information from the draft model and rendering the drafting process inefficient. The cache constructed by CARD capitalizes on the overlapping portions of the generation directions between the target model and draft model to cache the generation space of the target model. This allows for immediate retrieval of alternative sequences from the cache when a generated sequence is rejected, fully utilizing the historical information produced during each inference of the draft model.